

EECE 7205: Computing Fundamentals for Engineers

Fall 2026

In Class Work #01 Sorting Algorithms

Topic: Sorting Algorithm Analysis

Student Name	Eduardo Scopelli
NUID	003972121
Email	scopellirivas.e@northeastern.edu
Instructor	Dr. Fatema Nafa
Due Date	09-20-2026

Checklist Item

Goal: For each problem, complete the following steps.

- **Write Pseudocode:** Begin by drafting a pseudocode for each problem. This should outline the logic and steps your code will follow, presented in a simplified and understandable manner.
- **C++ Program:** Following the pseudocode, develop a C++ program that solves the problem. Ensure your code is clear, well-commented, and functional.
- **Implement the Algorithms:** Do not use `std::sort`, `std::stable sort`, or any other library sorting function. Every problem starts from the original unsorted array A.
- **Two Required Functions:** Every program must have (1) a function `printArray` that prints the array, and (2) a function that sorts it. Whenever a problem asks you to print the array, call `printArray`; do not write printing loops inside the sorting function or `main()`.

Code Preparation

Before starting with the sorting types and C++ programs / functions, a few utility functions will be built to aid in the testing and execution of these sorting algorithms. Firstly, a handful of helper functions will be defined for printing out arrays towards the console to be able to visualize the arrays within each type of algorithm. In this case, two functions were created: *printArray* which prints out an entire array's elements in string format, and *printSubArray* which prints only a section of an array (Used in the later algorithms which rely on splitting the original array). Additionally, we will create a function *subVector* to return a subsection of an array in array form, which will also be used in some of the later algorithms that require work on said sub-arrays.

Finally, a function *selectSortType* for executing the different algorithms will be created with a `split` statement in order to be able to select a type of algorithm to execute when running the code, following the assignments instructions to include the original unsorted array, execute the sorting algorithm and then print out the final sorted array.

```

string printArray(vector<int> A)
{
    string s = "[ ";
    for (int n : A) {
        s += to_string(n) + " ";
    }
    s += "]";
    return s;
}

string printSubArray(vector<int> A, int start, int end)
{
    string s = "[ ";
    for (int i = start; i <= end; i++) {
        s += to_string(A[i]) + " ";
    }
    s += "]";
    return s;
}

vector<int> subVector(vector<int> A, int start, int end)
{
    vector<int> B = {};
    for (int i = start; i <= end; i++) {
        B.push_back(A[i]);
    }

    return B;
}

void selectSortType(int p, vector<int> A)
{
    switch (p)
    {
        case 1: // Bubble Sort Algorithm
            cout << printArray(A) << endl;
            cout << printArray(bubbleSort(A)) << endl;
            break;
        case 2: // Insertion Sort Algorithm
            cout << printArray(A) << endl;
            cout << printArray(insertionSort(A)) << endl;
            break;
        case 3: // Selection Sort Algorithm
            cout << printArray(A) << endl;
            cout << printArray(selectionSort(A)) << endl;
            break;
        case 4: // Quick Sort Algorithm
            cout << printArray(A) << endl;
            quickSort(A, 0, A.size() - 1);
            cout << printArray(A) << endl;
            break;
        case 5: // Merge Sort Algorithm
            cout << printArray(A) << endl;
            mergeSort(A, 0, A.size() - 1);
            cout << printArray(A) << endl;
            break;
        case 6: // Heap Sort Algorithm
            cout << printArray(A) << endl;
            heapSort(A);
            cout << printArray(A) << endl;
            break;
        default:
            break;
    }
}

int main()
{
    vector<int> A = {34, 7, 23, 32, 5, 62, 14, 19};
    cout << "Select a Sorting Algorithm: ";
    int n = 0;
    cin >> n;
    cout << endl;

    selectSortType(n, A);

    return 0;
}

```

Problem 1: Bubble Sort

Description: Bubble Sort repeatedly steps through the array, compares each pair of adjacent elements, and swaps them if they are in the wrong order. After each pass, the largest remaining element moves to the end of the unsorted part.

Write Pseudocode

```

1  n ← SizeOf(A)
2
3  for i ← 0 to n - 2 do
4      swapped ← False
5      for j ← 0 to n - i - 2 do
6          if A[j] > A[j + 1] then
7              swap(A[j], A[j + 1])
8              swapped ← True
9          end if
10     end for
11     if swapped = False then
12         break
13     end if
14 end for
15
16 return A

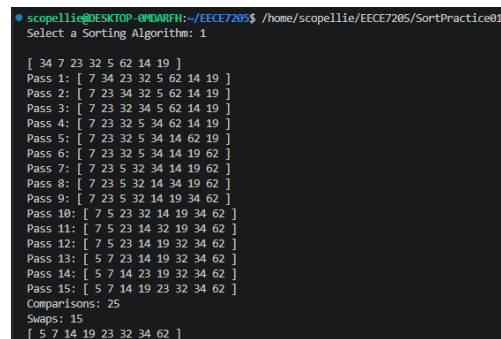
```

C++ Program

Function definition:

```
1 vector<int> bubbleSort(vector<int> A)
2 {
3     int n ← A.size();
4     int count_c ← 0; // Initialize count of comparisons
5     int count_s ← 0; // Initialize count of swaps
6
7     for (int i ← 0; i <← n - 2; i++) {
8         bool swapped ← false;
9         for (int j ← 0; j <← n - i - 2; j++) {
10             count_c++;
11             if (A[j] > A[j + 1]) {
12                 count_s++;
13                 int temp ← A[j]; // Swap array values using a temporal variable
14                 A[j] ← A[j + 1];
15                 A[j + 1] ← temp;
16                 swapped ← true;
17                 cout << "Pass " << count_s << ": " << printArray(A) << endl;
18             }
19         }
20         if (swapped == false) { // No swaps means the array is in order
21             break;
22         }
23     }
24
25     cout << "Comparisons: " << count_c << endl;
26     cout << "Swaps: " << count_s << endl;
27
28     return A;
29 }
```

Terminal output:



```
scopellie@DESKTOP-0NDANFH:~/EECE7205/SortPractice01$ ./bubbleSort.cpp
Select a Sorting Algorithm: 1
[ 34 7 23 32 5 62 14 19 ]
Pass 1: [ 7 34 23 32 5 62 14 19 ]
Pass 2: [ 7 23 34 32 5 62 14 19 ]
Pass 3: [ 7 23 32 34 5 62 14 19 ]
Pass 4: [ 7 23 32 5 34 62 14 19 ]
Pass 5: [ 7 23 32 5 34 14 62 19 ]
Pass 6: [ 7 23 32 5 34 14 19 62 ]
Pass 7: [ 7 23 5 32 34 14 19 62 ]
Pass 8: [ 7 23 5 32 14 34 19 62 ]
Pass 9: [ 7 23 5 32 14 19 34 62 ]
Pass 10: [ 7 5 23 32 14 19 34 62 ]
Pass 11: [ 7 5 23 14 32 19 34 62 ]
Pass 12: [ 7 5 23 14 19 32 34 62 ]
Pass 13: [ 5 7 23 14 19 32 34 62 ]
Pass 14: [ 5 7 14 23 19 32 34 62 ]
Pass 15: [ 5 7 14 19 23 32 34 62 ]
Comparisons: 25
Swaps: 15
[ 5 7 14 19 23 32 34 62 ]
```

As shown in the above image, executing the bubble sort for the provided array runs through 25 comparisons and completes 15 swaps of values. These 15 swaps can be seen in the terminal output as the "steps" completed for sorting this array. This large difference between comparisons and swaps can be explained by the nature of the bubble sort algorithm, which relies on comparing every pair of consecutive values, while only swapping them in the case that the first value is larger. Additionally,

this algorithm only comparing and swapping adjacent values in the array causes it to be prone to a larger amount of iterations if mid/larger values are present at the beginning of the array. For example, the value of 32 gets swapped 4 times consecutively before stopping at Pass 5 in which it runs into the value 62, even though it still hasn't reached its final position. This break for the value 32 causes the function to require an additional iteration of the full array before starting to move this value once again to its final position. Another way of seeing this is through the smaller numbers such as 5, which will require 4 outer iterations for it to reach its final position at the beginning of the array, as the swaps prioritize moving the larger values to the end of the array.

Time complexity: $O(n^2)$

Problem 2: Insertion Sort

Description: Insertion Sort builds the sorted part of the array one element at a time. It takes the next element (the key) and shifts every larger element in the sorted part one position to the right until the key can be inserted in its correct place.

Write Pseudocode

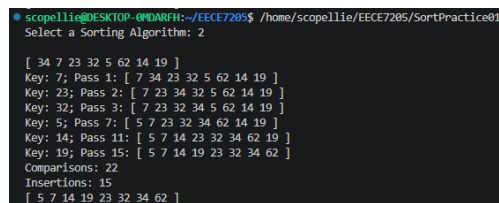
```
1 n ← SizeOf(A)
2
3 for i ← 1 to n - 1 do
4     key ← A[i]
5     j ← i - 1
6     while j > 0 and A[j] > key do
7         A[j + 1] ← A[j]
8         Decrement j
9     end while
10    if j ≠ i - 1 then
11        A[j + 1] ← key
12    end if
13 end for
14
15 return A
```

C++ Program

Function definition:

```
1 vector<int> insertionSort(vector<int> A)
2 {
3     int n ← A.size();
4     int count_c ← 0;
5     int count_s ← 0;
6
7     for (int i ← 1; i <= n - 1; i++) {
8         int key ← A[i]; // Define key value for each iteration
9         int j ← i - 1;
10        count_c++;
11
12        while (j >= 0 && A[j] > key)
13        {
14            count_c++;
15            count_s++;
16            A[j + 1] ← A[j]; // Shift larger values to the right
17            j--;
18        }
19        if (j ≠ i - 1) {
20            A[j + 1] ← key; // Insert key value into it's final position
21            cout << "Key: " << key << "; Pass " << count_s << ": " << printArray(A) <<
                endl;
22        }
23    }
24
25    cout << "Comparisons: " << count_c << endl;
26    cout << "Insertions: " << count_s << endl;
27
28    return A;
29 }
```

Terminal output:



```
scopellie@DESKTOP-0YDANFHK:~/ECE7205$ ./home/scopellie/ECE7205/SortPractice01
Select a Sorting Algorithm: 2

[ 34 7 23 32 5 62 14 19 ]
Key: 7; Pass 1: [ 7 34 23 32 5 62 14 19 ]
Key: 23; Pass 2: [ 7 23 34 32 5 62 14 19 ]
Key: 32; Pass 3: [ 7 23 32 34 5 62 14 19 ]
Key: 5; Pass 4: [ 5 7 23 32 34 62 14 19 ]
Key: 14; Pass 5: [ 5 7 14 23 32 34 62 19 ]
Key: 19; Pass 6: [ 5 7 14 19 23 32 34 62 ]
Comparisons: 22
Insertions: 15
[ 5 7 14 19 23 32 34 62 ]
```

As shown in the above image, executing the insertion sort for the provided array runs through 6 insertions in 6 passes, which involves 22 comparisons and 15 swaps. Compared to the bubble sort, this algorithm directly determines the final position for a value within the sorted section of the array at the beginning, shifting all larger values of the previously sorted section to the right to "insert" the new value, hence the number of insertions being given by the amount of out of order elements. For example, in the 4th pass where the key is the value 5, the function will compare this key with every value to the left until it finds a value that is smaller, assuring that all larger sorted values come after this one and all smaller values are maintained to the left. While the amount of insertions /

iterations is reduced with this algorithm. this method still requires all values larger than the key to be shifted to the right, adding more swap operations to the execution of this algorithm even though the number of passes is reduced.

Time complexity: $O(n^2)$

Problem 3: Selection Sort

Description: Selection Sort divides the array into a sorted part (on the left) and an unsorted part. In each pass it finds the smallest element in the unsorted part and swaps it into the first position of the unsorted part.

Write Pseudocode

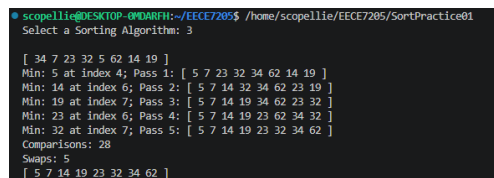
```
1 n ← SizeOf(A)
2
3 for i ← 1 to n - 2 do
4     min_i ← i
5     for j ← i to n - 1 do
6         if A[j] < A[min_i] then
7             min_i ← j
8         end if
9     end for
10    if i ≠ min_i then
11        swap(A[i], A[min_i])
12    end if
13 end for
14
15 return A
```

C++ Program

Function definition:

```
1 vector<int> selectionSort(vector<int> A)
2 {
3     int n ← A.size();
4     int count_c ← 0;
5     int count_s ← 0;
6
7     for (int i ← 0; i <← n - 2; i++) {
8         int min_i ← i;
9         for (int j ← i; j <← n - 1; j++) {
10             if (A[j] < A[min_i]) {
11                 count_c++;
12                 min_i ← j; // Determine index of smallest value
13             }
14         }
15         if (i ≠ min_i) {
16             int temp ← A[min_i]; // Swaps smallest value to beginning of the array
17             A[min_i] ← A[i];
18             A[i] ← temp;
19             count_s++;
20             cout << "Min: " << temp << " at index " << min_i << "; Pass " << count_s << ":
                " << printArray(A) << endl;
21         }
22     }
23
24     cout << "Comparisons: " << count_c << endl;
25     cout << "Swaps: " << count_s << endl;
26
27     return A;
28 }
```

Terminal output:



```
scopellie@DESKTOP-0MDARFH:~/EECE7205/SortPractice01$ ./selectionSort.cpp
Select a Sorting Algorithm: 3
[ 34 7 23 32 5 62 14 19 ]
Min: 5 at index 4; Pass 1: [ 5 7 23 32 34 62 14 19 ]
Min: 14 at index 6; Pass 2: [ 5 7 14 32 34 62 23 19 ]
Min: 19 at index 7; Pass 3: [ 5 7 14 19 34 62 23 32 ]
Min: 23 at index 6; Pass 4: [ 5 7 14 19 23 62 34 32 ]
Min: 32 at index 7; Pass 5: [ 5 7 14 19 23 32 34 62 ]
Comparisons: 28
Swaps: 5
[ 5 7 14 19 23 32 34 62 ]
```

As shown in the above image, executing the selection sort for the provided array completes only 5 swaps but 28 comparisons. Similar to the insertion sort, this algorithm will determine the final position of each element per iteration by finding the smallest value within the unsorted array. This logic causes the algorithm to need at most n swaps for any array, having to swap every element to its final position once without any additional swaps. Unfortunately, this algorithm does require a larger amount of comparisons, as with every iteration it has to go through every unsorted element of the array to determine which one is the smallest. For example, in the first iteration where the smallest value is 5 at index 4, the function will still go through the rest of the elements after 5 to determine that it is indeed the smallest value, before swapping it directly to the first spot. Due to this functionality, this algorithm will always have a larger amount

of comparisons such as with the bubble sort algorithm, but will require the smallest amount of swaps.

Time complexity: $O(n^2)$

Problem 4: Quick Sort

Description: Quick Sort is a divide-and-conquer algorithm. It picks a pivot, partitions the array so that smaller elements are on the pivot's left and larger elements are on its right, and then sorts the two sides recursively.

Write Pseudocode

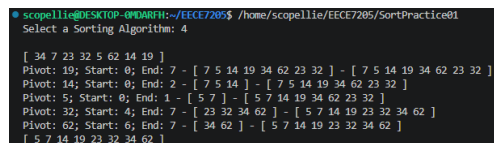
```
1 procedure quickSort(A, start, end)
2   if start < end then
3     pivot ← partition(A, start, end)
4
5     quickSort(A, start, pivot - 1)
6     quickSort(A, pivot + 1, end)
7   end if
8 end procedure
9
10 procedure partition(A, start, end)
11   i ← start - 1
12   pivot ← A[end]
13
14   for j ← start to end do
15     if A[j] < pivot then
16       Increment i
17       swap(A[j], A[i])
18     end if
19   end for
20
21   swap(A[end], A[i + 1])
22
23   return i + 1
24 end procedure
```

C++ Program

Function definition:

```
1 int partition(vector<int>& A, int start, int end)
2 {
3     int i ← start - 1;
4     int pivot ← A[end]; // Pivot selected at the end of the array
5
6     for (int j ← start; j < end; j++) {
7         if (A[j] < pivot) {
8             i++; // Determine final position for pivot value
9             int temp ← A[j];
10            A[j] ← A[i];
11            A[i] ← temp;
12        }
13    }
14
15    int temp ← A[end]; // Swap pivot value to it's corresponding position
16    A[end] ← A[i + 1];
17    A[i + 1] ← temp;
18
19    cout << "Pivot: " << pivot << "; Start: " << start << "; End: " << end << " - ";
20    cout << printSubArray(A, start, end) << " - " << printArray(A) << endl;
21
22    return i + 1;
23 }
24
25 void quickSort(vector<int>& A, int start, int end)
26 {
27     if (start < end) {
28         int pivot ← partition(A, start, end); // Determine pivot value to split array
29
30         quickSort(A, start, pivot - 1); // Split array along pivot and repeat process
31         quickSort(A, pivot + 1, end);
32     }
33 }
```

Terminal output:



```
scopellie@DESKTOP-0P3M0R8:~/EECE205/SortPractice01$
Select a Sorting Algorithm: 4
[ 34 7 23 32 5 62 14 19 ]
Pivot: 19; Start: 0; End: 7 - [ 7 5 14 19 34 62 23 32 ] - [ 7 5 14 19 34 62 23 32 ]
Pivot: 14; Start: 0; End: 2 - [ 7 5 14 ] - [ 7 5 14 19 34 62 23 32 ]
Pivot: 5; Start: 0; End: 1 - [ 5 7 ] - [ 5 7 14 19 34 62 23 32 ]
Pivot: 32; Start: 4; End: 7 - [ 23 32 34 62 ] - [ 5 7 14 19 23 32 34 62 ]
Pivot: 62; Start: 6; End: 7 - [ 34 62 ] - [ 5 7 14 19 23 32 34 62 ]
[ 5 7 14 19 23 32 34 62 ]
```

This is the first algorithm that uses recursion to sort an array in what is known as a "divide and conquer" algorithm. In this case, the algorithm uses a pivot point (in this case the last value) as a midpoint to split an array into two: one sub-array with values smaller to the pivot, and the other sub-array with values larger to the pivot. Once the array has been split into two sub-arrays, the same process is repeated with each sub-array until we reach arrays that can't be split any further. Additionally, this algorithm works using a direct memory reference to the original array definition, in which any swaps and changes made within the functions directly affects the original array definition,

taking away the need to return values or sub-arrays from the recurring function executions. We can see an example in the image above between the first and second step. In the first step we take the value of 19 as the pivot and move every element smaller to the left and keep all larger elements to the right. For the second step, we go down the first level of recursion, only working with the first sub-array of elements before the 19 and repeating the same process, which in turn affects the original array within the range of this sub-array. As an additional note, this algorithm could increase in time complexity to $O(n^2)$ if the selection of pivot points keeps taking the largest or smallest values in the array, causing each recursion to take in the same array just without the one pivot element.

Time complexity: $O(n \log(n))$

Problem 5: Merge Sort

Description: Merge Sort is a divide-and-conquer algorithm. It splits the array into two halves, sorts each half recursively, and then merges the two sorted halves into one sorted array.

Write Pseudocode

```

1 procedure mergeSort(A, low, high)
2   if low < high then
3     mid ← (low + high) / 2
4
5     mergeSort(A, low, mid)
6     mergeSort(A, mid + 1, high)
7
8     merge(A, low, mid, high)
9   end if
10 end procedure
11
12 procedure merge(A, low, high)
13   L ← A[low : mid]
14   R ← A[mid + 1 : high]
15
16   while l < SizeOf(L) and r < SizeOf(R) do
17     if L[l] < R[r] then
18       A[a] ← L[l]
19       Increment l
20       Increment a
21     else
22       A[a] ← R[r]
23       Increment r
24       Increment a
25     end if
26   end while
27
28   while l < SizeOf(L) do
29     A[a] ← L[l]
30     Increment l
31     Increment a
32   end while

```

```

33
34     while r < SizeOf(R) do
35         A[a] ← R[r]
36         Increment r
37         Increment a
38     end while
39 end procedure

```

C++ Program

Function definition:

```

1 void merge(vector<int>& A, int low, int mid, int high)
2 {
3     vector<int> L ← subVector(A, low, mid); // Use helper function to split array
4     vector<int> R ← subVector(A, mid + 1, high);
5
6     int l ← 0; int r ← 0; int a ← low; // Initialize pointers per array
7
8     while (l < L.size() && r < R.size()) { // Merge arrays L and R by smallest values
9         if (L[l] < R[r]) {
10             A[a] ← L[l];
11             l++; a++;
12         } else {
13             A[a] ← R[r];
14             r++; a++;
15         }
16     }
17     while (l < L.size()) { // Add remaining values in arrays L / R
18         A[a] ← L[l];
19         l++; a++;
20     }
21     while (r < R.size()) {
22         A[a] ← R[r];
23         r++; a++;
24     }
25 }
26
27 void mergeSort(vector<int>& A, int low, int high)
28 {
29     cout << "Sub-array: " << printSubArray(A, low, high) << " - " << printArray(A) <<
        endl;
30     if (low < high) {
31         int mid ← (low + high) / 2;
32         mergeSort(A, low, mid); // Split input array into two
33         mergeSort(A, mid + 1, high);
34         merge(A, low, mid, high); // Merge split arrays
35         cout << "Merged array: " << printSubArray(A, low, high) << " - " << printArray(A)
            << endl;
36     }
37 }

```

Terminal output:

```
scopellie@DESKTOP-9MD9RFH:~/ECE7205$ ./home/scopellie/ECE7205/SortPractice01
Select a Sorting Algorithm: 5

[ 34 7 23 32 5 62 14 19 ]
Sub-array: [ 34 7 23 32 5 62 14 19 ] - [ 34 7 23 32 5 62 14 19 ]
Sub-array: [ 34 7 23 32 ] - [ 34 7 23 32 5 62 14 19 ]
Sub-array: [ 34 7 ] - [ 34 7 23 32 5 62 14 19 ]
Sub-array: [ 34 ] - [ 34 7 23 32 5 62 14 19 ]
Sub-array: [ 7 ] - [ 34 7 23 32 5 62 14 19 ]
Merged array: [ 7 34 ] - [ 34 7 23 32 5 62 14 19 ]
Sub-array: [ 23 32 ] - [ 7 34 23 32 5 62 14 19 ]
Sub-array: [ 23 ] - [ 7 34 23 32 5 62 14 19 ]
Sub-array: [ 32 ] - [ 7 34 23 32 5 62 14 19 ]
Merged array: [ 23 32 ] - [ 7 34 23 32 5 62 14 19 ]
Merged array: [ 7 23 32 34 ] - [ 7 23 32 34 5 62 14 19 ]
Sub-array: [ 5 62 14 19 ] - [ 7 23 32 34 5 62 14 19 ]
Sub-array: [ 5 62 ] - [ 7 23 32 34 5 62 14 19 ]
Sub-array: [ 5 ] - [ 7 23 32 34 5 62 14 19 ]
Sub-array: [ 62 ] - [ 7 23 32 34 5 62 14 19 ]
Merged array: [ 5 62 ] - [ 7 23 32 34 5 62 14 19 ]
Sub-array: [ 14 19 ] - [ 7 23 32 34 5 62 14 19 ]
Sub-array: [ 14 ] - [ 7 23 32 34 5 62 14 19 ]
Sub-array: [ 19 ] - [ 7 23 32 34 5 62 14 19 ]
Merged array: [ 14 19 ] - [ 7 23 32 34 5 62 14 19 ]
Merged array: [ 5 14 19 62 ] - [ 7 23 32 34 5 14 19 62 ]
Merged array: [ 5 7 14 19 23 32 34 62 ] - [ 5 7 14 19 23 32 34 62 ]
[ 5 7 14 19 23 32 34 62 ]
```

This algorithm utilizes similar logic to the quick sort algorithm, splitting up the original array into parts and building it back together. In this case however, the array is split directly in half based on the size until these sub-arrays have only one element each. Once we reach this point, the sub-arrays are merged back together, where we compare each of the smaller values in each sub-array to determine which element comes first in the original array. This process can be seen in the image above, which shows the steps of splitting down the original array into smaller sub-arrays until it reaches single element arrays, and then merging them back together in order. This back and forth eventually results in merging back a fully sorted array, with each merge step re-sorting the two sub-arrays being merged. As this algorithm always splits down the arrays in half, it truly achieves the "divide and conquer" logic, truly achieving a better time complexity in any case.

Time complexity: $O(n\log(n))$

Problem 6: Heap Sort

Description: Heap Sort stores the array as a binary max-heap, where every parent is greater than or equal to its children and the children of index k are at $2k + 1$ and $2k + 2$. It first builds the heap, then repeatedly swaps the maximum (the root) to the end of the array and restores the heap on the remaining elements.

Write Pseudocode

```
1 procedure heapSort(A)
2   n ← SizeOf(A)
3
4   for i ← (n / 2) - 1 to 0 do
5     heapify(A, n, i)
6   end for
7
8   for i ← n - 1 to 0 do
9     swap(A[0], A[i])
10    heapify(A, i, 0)
11  end for
12 end procedure
```

```

13
14 procedure heapify(A, n, i)
15     max ← i
16     left ← 2i + 1
17     right ← 2i + 2
18
19     if left < n and A[max] < A[left] then
20         max ← left
21     end if
22     if right < n and A[max] < A[right] then
23         max ← right
24     end if
25
26     if max ≠ i then
27         swap(A[i], A[max])
28         heapify(A, n, max)
29     end if
30 end procedure

```

C++ Program

Function definition:

```

1 void heapify(vector<int>& A, int n, int i)
2 {
3     int max ← i; // Index values for a parent - children pair in a binary tree
4     int left ← 2 * i + 1;
5     int right ← 2 * i + 2;
6
7     if (left < n && A[left] > A[max]) {
8         max ← left;
9     }
10    if (right < n && A[right] > A[max]) {
11        max ← right;
12    }
13    if (max ≠ i) { // Move largest element to parent position (max heap constraint)
14        int temp ← A[i];
15        A[i] ← A[max];
16        A[max] ← temp;
17
18        heapify(A, n, max); // Move down binary tree
19    }
20 }
21
22 void heapSort(vector<int>& A){
23     int n ← A.size();
24
25     for (int i ← n / 2 - 1; i >= 0; i--) { // Create initial max-heap
26         heapify(A, n, i);
27     }
28     cout << "Initial max heap: " << printArray(A) << endl;
29
30     for (int i ← n - 1; i >= 0; i--) {

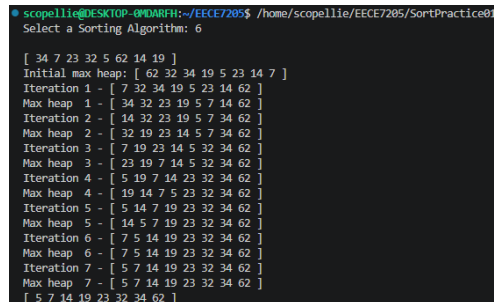
```

```

31     cout << "Iteration " << n - i << " - " << printArray(A) << endl;
32     int temp ← A[0]; // Move largest value to the end
33     A[0] ← A[i];
34     A[i] ← temp;
35     heapify(A, i, 0);
36 }
37 }

```

Terminal output:



```

scopellie@DESKTOP-QNDARFH:~/EECE7205/SortPractice01
Select a Sorting Algorithm: 6

[ 34 7 23 32 5 62 14 19 ]
Initial max heap: [ 62 32 34 19 5 23 14 7 ]
Iteration 1 - [ 7 32 34 19 5 23 14 62 ]
Max heap 1 - [ 34 32 23 19 5 7 14 62 ]
Iteration 2 - [ 14 32 23 19 5 7 34 62 ]
Max heap 2 - [ 32 19 23 14 5 7 34 62 ]
Iteration 3 - [ 7 19 23 14 5 32 34 62 ]
Max heap 3 - [ 23 19 7 14 5 32 34 62 ]
Iteration 4 - [ 5 19 7 14 23 32 34 62 ]
Max heap 4 - [ 19 14 7 5 23 32 34 62 ]
Iteration 5 - [ 5 14 7 19 23 32 34 62 ]
Max heap 5 - [ 14 5 7 19 23 32 34 62 ]
Iteration 6 - [ 7 5 14 19 23 32 34 62 ]
Max heap 6 - [ 7 5 14 19 23 32 34 62 ]
Iteration 7 - [ 5 7 14 19 23 32 34 62 ]
Max heap 7 - [ 5 7 14 19 23 32 34 62 ]
[ 5 7 14 19 23 32 34 62 ]

```

This final algorithm utilizes the Max Heap data structure to sort the elements of an array. A Max Heap is a kind of binary tree in which every parent node is larger than its children nodes, which for a tree with multiple levels causes the largest possible numbers to always be at the top, while the smallest numbers stay at the bottom or at the end of each branch. In this case, having the largest number of the original array as the parent node in the first index means we can immediately determine the value that belongs at the end of our expected sorted array. Thus, the heap sort algorithm consists of two main steps: Transforming the array into a max heap, and moving the largest value to the end of the array. The image above demonstrates these steps, in which for each iteration the largest element is moved to the last position of the array, and the rest of the array is structured back into a max heap before moving to the next iteration. Comparing this algorithm to the previous two which also run in $O(n \log(n))$ time, heap sort maintains the same time complexity from merge sort in every case, but works with its array "in-place" like the quick sort algorithm, maintaining a lower space complexity of $O(n)$ based solely on the input array, unlike the merge sort algorithm which requires additional auxiliary arrays.

Time complexity: $O(n \log(n))$

Problem 7: Comparing the Six Algorithms

Description: Bubble Sort, Insertion Sort, and Selection Sort take $O(n^2)$ time in the worst case, while Merge Sort and Heap Sort take $O(n \log(n))$. Quick Sort takes $O(n \log(n))$ on average but $O(n^2)$ in its worst case. In this problem you will check these results with your own code.

C++ Program

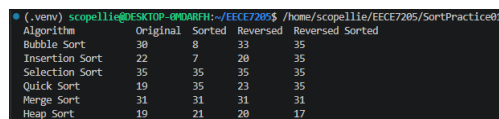
Function definition:

```

1 int main()
2 {
3     ...
4     // Code for counting and printing the number of iterations / comparisons completed
      for each algorithm using the base array A:
5     cout << left << setw(20) << "Algorithm" << left << setw(10) << "Original" << left <<
      setw(8) << "Sorted" << left << setw(10) << "Reversed" << left << setw(17) << "
      Reversed Sorted" << endl;
6     for (int i ← 1; i <← 6; i++) {
7         compareAlgorithms(A, i, true);
8     }
9     cout << endl;
10    // Function for executing each algorithm with 1000, 5000 and 10000 element arrays and
      counting execution time
11    timeAnalysis();
12    return 0;
13 }

```

Terminal output (Iteration / Comparison Counter):



Algorithm	Original	Sorted	Reversed	Reversed Sorted
Bubble Sort	30	8	33	35
Insertion Sort	22	7	20	35
Selection Sort	35	35	35	35
Quick Sort	19	35	23	35
Merge Sort	31	31	31	31
Heap Sort	19	21	20	17

This first image shows a table with the amount of iterations / comparisons completed by each type of algorithm with four copies of the original array A sorted in different ways: Original, Sorted, Reversed and Reversed Sorted. With this result we can compare the possible effects that the orientation of the input array may have on each type of algorithm. First, both the Selection Sort and Merge Sort algorithms appear to not be affected by the change in orientation of the input array, due to the fact that both algorithms compare or iterate over every element in the array even if certain sections are already partially sorted. Moving on with the first three algorithms that have a time complexity of $O(n^2)$, we notice that both the Bubble Sort and Insertion Sort have their best case scenario when the array is already sorted, and worst case when it is sorted in reverse. This first observation demonstrates that these two algorithms are ideal for already sorted arrays due to their validation of this edge case. Additionally, we can see that the Insertion Sort algorithm is slightly faster than the Bubble Sort algorithm in essentially every case, meaning that of the first three algorithms Insertion Sort should be the fastest. Finally, with the last three algorithms that have a time complexity of $O(n \log(n))$ we notice similar behaviors to those mentioned previously. Firstly it is important to note that the Quick Sort algorithm does struggle the most when the array is already sorted due to our pivot selection, which in these cases would always select either the smallest or largest value and thus, force the function to fully iterate over every element of the array. When comparing these last three algorithms, Heap Sort appears to be slightly faster than the previous two, meaning that it should also be the fastest out of these three according to these results. As a final comment on this section, the method for counting these iterations or comparisons within each algorithm was completed by adding a counter variable into each and incrementing this variable with every iteration of a loop or with every recursive call of a function. With this in mind, it is important not to take these shown results as the absolute truth about the efficiency of these algorithms, but instead as a helpful pointer for the behavior of them with different inputs.

Terminal output (Execution Time):

```
Time analysis (1000 elements):
Bubble Sort - 5204 us - [ 81 237 346 545 569 606 657 681 723 925 973 ]
Insertion Sort - 1733 us - [ 81 237 346 545 569 606 657 681 723 925 973 ]
Selection Sort - 2681 us - [ 81 237 346 545 569 606 657 681 723 925 973 ]
Quick Sort - 122 us - [ 81 237 346 545 569 606 657 681 723 925 973 ]
Merge Sort - 612 us - [ 81 237 346 545 569 606 657 681 723 925 973 ]
Heap Sort - 218 us - [ 81 237 346 545 569 606 657 681 723 925 973 ]

Time analysis (5000 elements):
Bubble Sort - 120727 us - [ 10 30 34 44 77 93 96 113 117 131 178 ]
Insertion Sort - 44449 us - [ 10 30 34 44 77 93 96 113 117 131 178 ]
Selection Sort - 70694 us - [ 10 30 34 44 77 93 96 113 117 131 178 ]
Quick Sort - 807 us - [ 10 30 34 44 77 93 96 113 117 131 178 ]
Merge Sort - 5624 us - [ 10 30 34 44 77 93 96 113 117 131 178 ]
Heap Sort - 1386 us - [ 10 30 34 44 77 93 96 113 117 131 178 ]

Time analysis (10000 elements):
Bubble Sort - 522631 us - [ 4 13 14 29 44 64 154 158 161 168 179 ]
Insertion Sort - 185581 us - [ 4 13 14 29 44 64 154 158 161 168 179 ]
Selection Sort - 269823 us - [ 4 13 14 29 44 64 154 158 161 168 179 ]
Quick Sort - 1717 us - [ 4 13 14 29 44 64 154 158 161 168 179 ]
Merge Sort - 16368 us - [ 4 13 14 29 44 64 154 158 161 168 179 ]
Heap Sort - 3393 us - [ 4 13 14 29 44 64 154 158 161 168 179 ]
```

This final image demonstrates the execution time of each algorithm when sorting random arrays of 1,000, 5,000 and 10,000 elements. From this result we can observe the growth rate of each type of algorithm, confirming that the first three algorithms do demonstrate an exponential growth when the number of elements to sort is increased. On the other hand, the last three algorithms do show a lower growth rate compared to the first three, which supports the lower time complexity scale that these three algorithms have. Finally, when comparing the execution times themselves, it seems that the Quick Sort and Heap Sort are the two most efficient algorithms for sorting large random arrays according to this test, with the Merge Sort algorithm not far behind.